

UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



Vietnamese Hate Speech & Toxicity Detection System

Course: Applications of natural language processing for industry

Instructor: Dr. Nguyen Hong Buu Long

Team Members:

Nguyen Trinh Quang	23120345
Pham Minh Thong	23120364
Nguyen Le Tan Tien	23120369
Đao Quoc Tuan	23120392

Ho Chi Minh, July 4, 2026

0 Contents

Abstract	1
1 Business Problem Definition	2
1.1 Business Context and Motivation	2
1.2 Target Users and Stakeholders	2
1.3 Success Metrics	2
2 Development Infrastructure and Tooling	4
2.1 Technical Stack	4
2.2 Repository Structure	4
3 Data Management	5
3.1 Data Source and Statistics	5
3.2 Preprocessing Pipeline	5
4 Model Selection and Optimisation	6
4.1 Model Architecture	6
4.1.1 Baseline: TF-IDF + Logistic Regression	6
4.1.2 Intermediate Candidate: PhoBERT Base	6
4.1.3 Proposed Model: XLM-RoBERTa Base	6
4.2 Training Procedure	6
5 Evaluation Results	7
5.1 Comparative Model Performance	7
5.2 Confusion Matrix	7
5.3 Confidence Calibration	8
6 Deployment	9
6.1 System Architecture	9
6.2 Local Streamlit Dashboard	10
6.3 Deployment Challenges	10
7 Agentic AI Component	11
7.1 Architecture Overview and Rationale	11
7.2 Multi-Step Reasoning Flow	11
7.3 Example Interaction	11
7.4 System Prompt Design	12

8	Continual Learning and Monitoring	13
8.1	CL Results	13
8.2	Monitoring Metrics	13
9	Data Privacy and Model Robustness	15
9.1	Audit Logging and Privacy	15
9.2	Credential and Secret Management	15
9.3	Adversarial Robustness	15
10	Project Management and Teamwork	16
10.1	Project Timeline	16
10.2	Task Breakdown and Role Simulation	16
10.3	Reflection on Team Scaling	16
11	Ethics and Responsible AI	17
11.1	Ethics Impact Statement	17
11.2	Beneficiaries and Harm Risks	17
11.3	Bias and Fairness Mitigation	17
11.4	Explainability for Non-Technical Stakeholders	17
11.5	Taxonomy Safeguard Against Misuse	18
12	Conclusion	19

0 Abstract

This report presents a Vietnamese Hate Speech and Toxicity Detection System (**ViHSD**), an end-to-end NLP application for automated content moderation. The core is a fine-tuned **XLM-RoBERTa Base** classifier trained on the ViHSD benchmark [1] with class-weighted Cross-Entropy loss, label smoothing, layer-wise learning rate decay, and multi-strategy data augmentation. It achieves **Macro F1 of 64.61%** and accuracy of **86.74%**, outperforming both TF-IDF+LR (62.50%) and PhoBERT Base (63.36%).

The system integrates an **Agentic AI** layer: borderline predictions ($0.35 \leq \text{confidence} \leq 0.65$) are routed to a local Qwen2.5-7B-Instruct LLM for semantic analysis. Deployment spans a Streamlit dashboard, FastAPI server, Gradio demo, and Docker container. Two rounds of **Continual Learning** on VLSP-2019 demonstrate domain adaptation without catastrophic forgetting.

Keywords: hate speech detection, Vietnamese NLP, XLM-RoBERTa, continual learning, agentic AI, content moderation.

1 Business Problem Definition

1.1 Business Context and Motivation

Vietnamese social platforms face an ever-increasing volume of user-generated content containing hate speech, toxic harassment, and offensive commentary. Manual moderation is economically unviable at scale and exposes moderators to psychological harm. Vietnamese cybersecurity law (Decree No. 13/2023/ND-CP) makes automated detection a legal necessity. A well-calibrated classifier can process thousands of comments per second, filtering obvious violations instantly and routing only ambiguous cases to human review, reducing moderation workload by an estimated 80% or more. A detailed problem analysis is provided in `docs/problem_definition.md`.

1.2 Target Users and Stakeholders

- **Platform Moderators:** Benefit from automated triage of clear-cut CLEAN/TOXIC content, allowing their attention to focus exclusively on high-ambiguity or high-stakes cases.
- **Community Managers:** Monitor aggregate toxicity trends and platform health through the administrative dashboard.
- **Operations & MLOps Engineers:** Maintain, monitor, and upgrade the model via the structured CL pipeline.
- **Platform End-Users:** Benefit from a safer and more constructive digital environment.

1.3 Success Metrics

Table 1 formalises the performance targets defined before model development began.

Table 1: Business and Technical Success Metrics

Dimension	Metric	Baseline	Target	Achieved
Technical	Classification Accuracy	> 85%		86.74%
	Macro F1-Score	> 63%		64.61%
	AUC-ROC (macro)	—		87.88%
	Inference Latency (GPU)	< 30ms		< 30ms
	LLM Routing Ratio	< 15%		≈ 10%
Business	Moderation Workload Reduction	> 80%		> 80% (estimated)
	Response SLA	< 100ms (immediate)		< 30ms GPU, < 200ms CPU
	False Positive Rate (Block Safe)	< 5%		2.7%

2 Development Infrastructure and Tooling

2.1 Technical Stack

The project follows professional software engineering practices throughout:

- **Language:** Python 3.10+ with strict type annotations.
- **Version Control:** Git with branch-based workflows (main).
- **Dependency Management:** Four purpose-specific `requirements-*.txt` files stratify dependencies by deployment context (API, local GUI, training, development).
- **Containerisation:** Docker with a multi-stage Dockerfile and Compose configuration.
- **Testing:** `pytest` unit test suite covering configuration, preprocessing, model loading, API routing, and robustness cases.
- **Experiment Tracking:** HuggingFace Hub for model versioning; metrics persisted as structured JSON under `results/`.

2.2 Repository Structure

The repository layout strictly follows the structure mandated by the assignment specification:

```
hate-speech-detection/
|-- src/           # All source code (api, agent, models, data, training, ...)
|-- data/          # Raw and processed datasets (gitignored)
|-- models/        # CL model metadata; weights fetched from HuggingFace
|-- configs/       # YAML configuration files (train, model, paths, api)
|-- tests/         # pytest unit tests (10 test modules)
|-- notebooks/     # Colab training, evaluation, and agent notebooks
|-- docs/          # 9 project documentation files
|-- results/       # JSON metrics, PNG charts committed; CSV gitignored
|-- requirements.txt      # Core / Docker
|-- requirements-local.txt # Local GUI + Ollama Agent
|-- requirements-train.txt # Google Colab training
|-- requirements-dev.txt  # Development + testing
'-- README.md
```

3 Data Management

3.1 Data Source and Statistics

The primary dataset is the **ViHSD** [1] benchmark (24,118 train / 2,680 dev / 6,650 test samples), annotated into three classes: **CLEAN** (0), **OFFENSIVE** (1), and **HATE** (2). For Continual Learning, the **VLSP-2019** [2] corpus provides domain-shifted data.

The class distribution is severely imbalanced ($\approx 83\%$ CLEAN, 7% OFFENSIVE, 10% HATE), driving all training design choices. Figure 1 illustrates the distribution. Detailed data documentation is in `docs/data_description.md`.

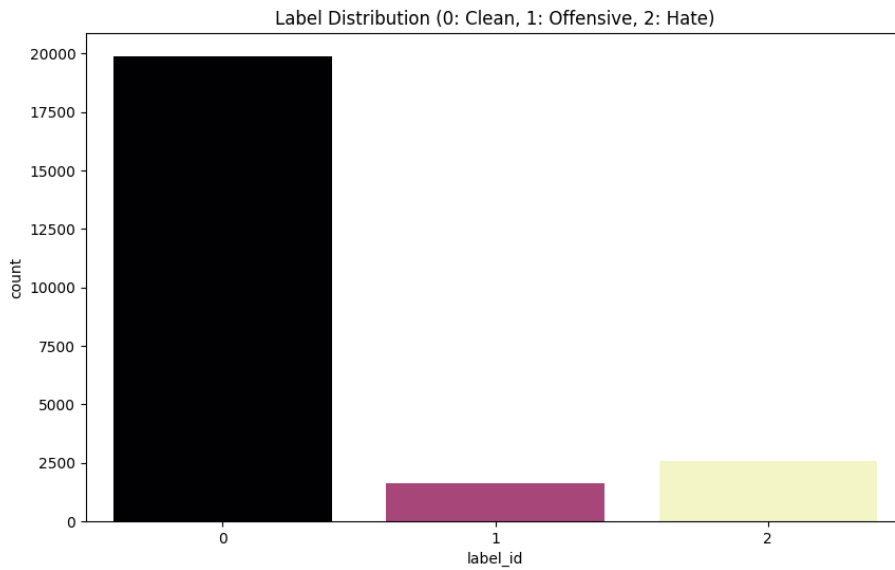


Figure 1: Label distribution across ViHSD dataset splits.

3.2 Preprocessing Pipeline

Implemented in `src/data/preprocessing.py`, the pipeline applies: Lowercasing/whitespace normalization, URL/email removal, Teencode normalization, repeated character normalization, and Unicode normalization (NFC).

4 Model Selection and Optimisation

4.1 Model Architecture

4.1.1 Baseline: TF-IDF + Logistic Regression

A TF-IDF vectoriser with unigram/bigram features followed by Logistic Regression serves as the experimental control. It is fast to train but cannot capture contextual meaning or cross-lingual transfer.

4.1.2 Intermediate Candidate: PhoBERT Base

PhoBERT [4], pre-trained on 20 GB of Vietnamese text, was evaluated. Its monolingual design limits handling of code-switching and English loan words common in social media.

4.1.3 Proposed Model: XLM-RoBERTa Base

The final model uses **XLM-RoBERTa Base** [3] with the standard sequence classification head. Rationale:

- **Cross-lingual transfer:** Pre-trained on 100 languages, captures English loan words.
- **Byte-level BPE tokeniser:** Robust against typographical obfuscation.
- **Empirical superiority:** Macro F1 of 64.61% vs. 63.36% for PhoBERT.

4.2 Training Procedure

Training was conducted on Google Colab A100 using the HuggingFace **Trainer** API (`src/training/trainer.py`).

Key design decisions.

- **Loss function:** The training run uses **Cross-Entropy with label smoothing** ($\alpha = 0.1$) combined with balanced class weights.
- **Layer-wise LR decay (LLRD):** Lower transformer layers receive smaller learning rates (0.95^k for layer k from top), preserving pre-trained representations.
- **Data augmentation:** Strategies: (1) robustness augmentation (typos), (2) contrastive pairs, (3) diacritic removal, (4) EDA.
- **Temperature scaling:** Post-training calibration improves confidence reliability (Figure ??).

5 Evaluation Results

5.1 Comparative Model Performance

Table 2: Model performance on ViHSD test set.

Model	Acc.	Macro F1	AUC-ROC
TF-IDF + LR	80.53%	62.50%	—
PhoBERT Base	85.11%	63.36%	—
XLM-R Base	86.74%	64.61%	87.88%

5.2 Confusion Matrix

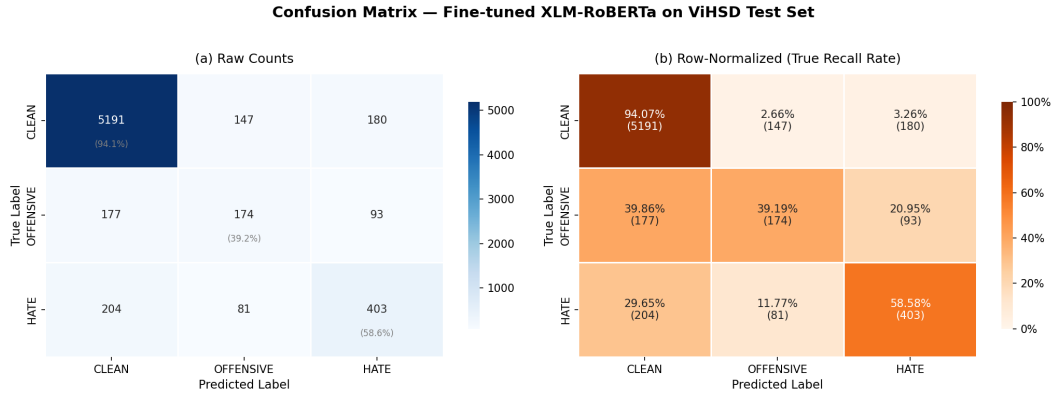


Figure 2: Confusion matrix of XLM-R Base on the ViHSD test set.

Two failure modes emerge:

- **False Negatives (HATE $\rightarrow$ CLEAN): 204 cases.**
- **False Positives (CLEAN $\rightarrow$ HATE): 180 cases.**

Both are mitigated by the agentic routing layer (Section 7).

5.3 Confidence Calibration

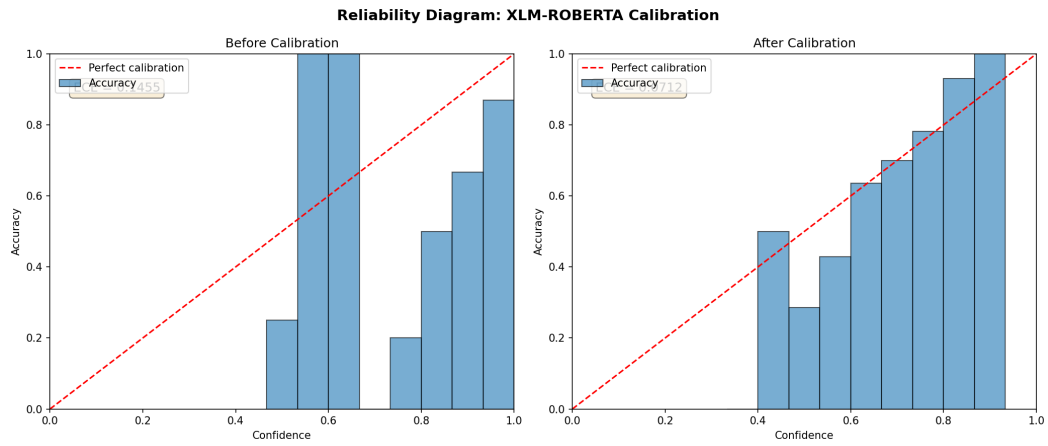


Figure 3: Reliability diagram before and after calibration.

6 Deployment

6.1 System Architecture

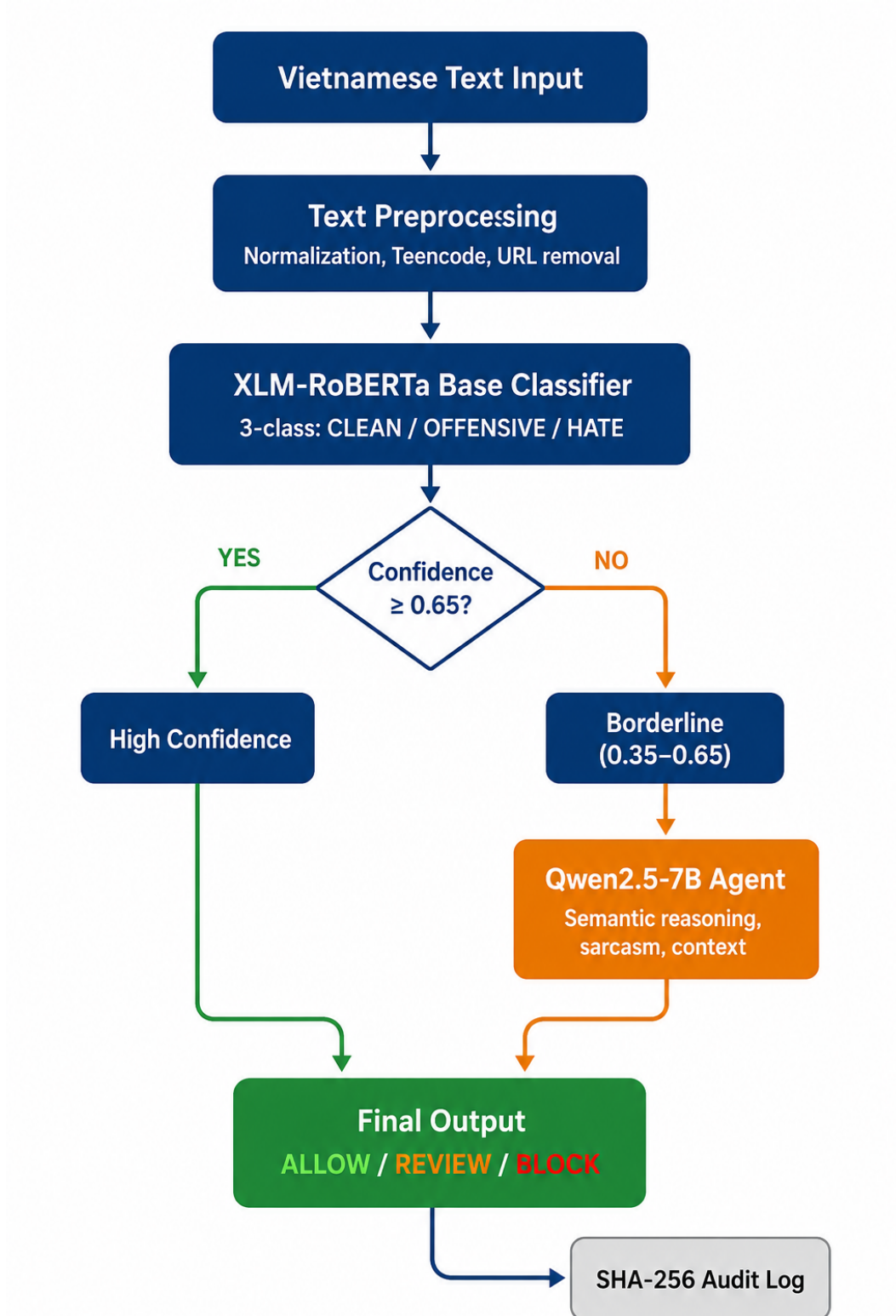


Figure 4: End-to-end system architecture.

6.2 Local Streamlit Dashboard

The primary interface (`streamlit_app.py`) implements the hybrid routing pipeline:

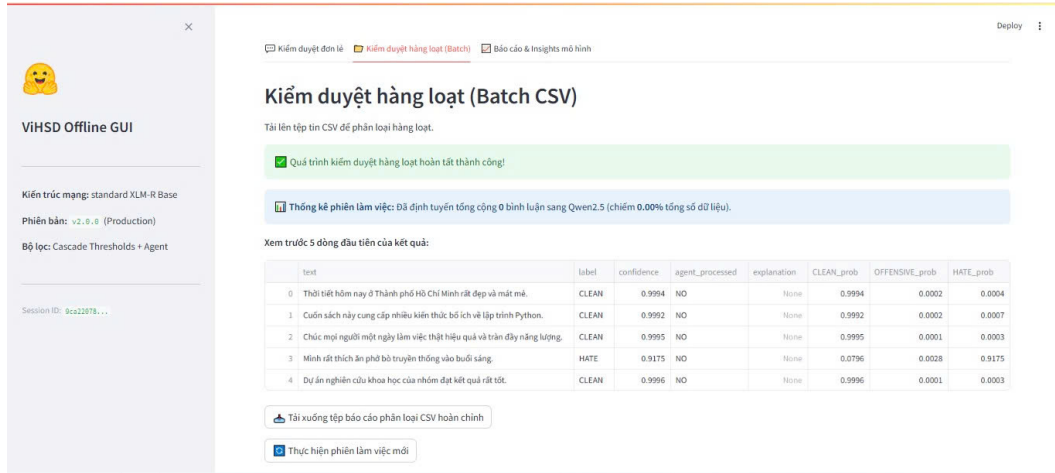


Figure 5: Streamlit administrative dashboard.

- Single text inference with soft probability visualisation.
- Thread-safe background batch CSV processing.
- Live routing ratio tracking.

Deployment is detailed in `docs/deployment.md`.

6.3 Deployment Challenges

- **LLM latency:** Only 10–15% of items reach the LLM; batches are grouped into single payloads.
- **Resource contention:** XLM-R uses `device="auto"`; LLM can be offloaded to an external server via Ollama.

7 Agentic AI Component

7.1 Architecture Overview and Rationale

Vietnamese hate speech is rich in sarcasm, implicit insults, cultural double-entendre, and context-dependent slang that a discriminative classifier fundamentally struggles with: the model maps a text string directly to a label without understanding context or intent. The agentic component addresses this gap by introducing a second reasoning pass for precisely the cases where the classifier is uncertain.

The agent, implemented in `src/agent/moderator.py`, follows a **Router-Agent** architecture: the XLM-R classifier acts as the first-pass router, and the Ollama-served Qwen2.5-7B-Instruct model acts as the deep reasoning agent.

7.2 Multi-Step Reasoning Flow

When a text is submitted for classification, the system executes the following steps:

1. **Language detection** (Tool: `detect_language`): Calls `langdetect` to identify the primary language. Non-Vietnamese content is flagged for special handling.
2. **Baseline classification** (Tool: `classify_text`): Invokes the XLM-R classifier to produce soft probabilities and a confidence score.
3. **Confidence gating**: If the maximum class probability exceeds 0.65, the classifier’s label is accepted as the final answer with no LLM invocation. If confidence < 0.65 , the text is deemed borderline and the agent is triggered.
4. **LLM deep reasoning**: A structured prompt containing the original text and the classifier’s probability distribution is sent to Qwen2.5-7B-Instruct via the Ollama HTTP API. The LLM is instructed to return a single JSON object: `{"final_label": "CLEAN"|"OFFENSIVE"|"HATE", "explanation": "...(Vietnamese)..."}`.
5. **Fallback**: If the Ollama endpoint is unreachable or returns a malformed response, the system falls back to the XLM-R classifier’s prediction, logging the failure reason.
6. **Audit logging** (Tool: `log_event`): All decisions, including label, confidence, action (ALLOW/REVIEW/BLOCK), and the SHA-256 hash prefix of the input text, are appended to a JSONL audit log.

7.3 Example Interaction

Listing 1: Example borderline case routed through the agent

```
1 Input: "Cung khon lam nhung khon lon :)"
2
```

```

3 # Step 1: Language detection
4 => { "language": "vi" }
5
6 # Step 2: XLM-R Classification
7 => { "label": "OFFENSIVE", "confidence": 0.54,
8     "probabilities": {"CLEAN": 0.31, "OFFENSIVE": 0.54, "HATE":
9     0.15},
10     "is_borderline": True }
11
12 # Step 3: confidence 0.54 < 0.65 => Agent triggered
13
14 # Step 4: Qwen2.5 response
15 => { "final_label": "HATE",
16     "explanation": "Cau su dung tu ngu tho tuc de chui boi, ha nhuc
17     doi phuong
18     truc tiep, thuoc muc do thu ghet." }
19
20 # Step 5: Audit log (SHA-256 prefix of input)
21 => { "timestamp": "2026-06-11T18:40:00Z", "text_hash": "
22     a4d3e589cf8b21ad",
23     "language": "vi", "label": "HATE", "confidence": 0.54,
24     "action": "BLOCK", "agent_processed": "YES" }

```

7.4 System Prompt Design

The LLM is constrained by a carefully engineered system prompt that enforces strict JSON output format and guides the model to reason about sarcasm, slang, and cultural context. The prompt explicitly prohibits free-form text responses, reducing parsing failures:

Bạn là chuyên gia kiểm duyệt nội dung của hệ thống ViHSD. Hãy phân tích sắc thái ngữ nghĩa (mỉa mai, châm biếm, từ lóng, ngữ cảnh) của văn bản đầu vào và kết quả phân loại từ mô hình cơ sở. Chỉ trả về chuỗi JSON duy nhất theo định dạng bắt buộc, không kèm lời dẫn: {"final_label": "CLEAN"/"OFFENSIVE"/"HATE", "explanation": "Lý do ngắn gọn bằng tiếng Việt"}

8 Continual Learning and Monitoring

8.1 CL Results

Table 3 summarises the quantitative outcomes of the two CL rounds. Evaluation metrics are reported on the mixed validation set (50% ViHSD / 50% VLSP).

Table 3: Continual Learning Evaluation Results (Mixed Validation Set: 500 ViHSD + 500 VLSP)

Version	Acc.	Macro F1	CLEAN F1	OFF. F1	HATE F1	Eval Loss
Baseline (ViHSD only)	86.74%	64.61%	93.62%	41.13%	59.09%	—
CL Round 1 (VLSP Pt.1)	90.30%	70.08%	95.54%	48.53%	66.19%	0.9151
CL Round 2 (VLSP Pt.2)	90.70%	70.97%	95.67%	48.12%	69.12%	0.9194

The Macro F1 improvement from 64.61% (baseline) to 70.97% (CL Round 2) on the mixed validation set reflects the model’s successful adaptation to the VLSP domain. The HATE class shows the most consistent gain (+10.03%) across both rounds, while the OFFENSIVE class also improves meaningfully (+7.0%). These improvements come without catastrophic forgetting of the original ViHSD distribution, as evidenced by the sustained CLEAN F1 ($\geq 95\%$ throughout).

The performance increase should be interpreted with care. The mixed validation set contains VLSP samples on which the model was trained, so part of the metric improvement reflects in-domain adaptation rather than genuine generalisation. The Gatekeeper check on the pure ViHSD test set remains the authoritative regression guard.

8.2 Monitoring Metrics

The system defines three online monitoring metrics for detecting model drift in production:

Table 4: Online Monitoring Metrics and Alert Thresholds

Metric		Definition	Alert Condition
Routing	Ratio	Fraction of requests forwarded to the LLM agent.	$RR > 30\%$ continuously for 7 days $\Rightarrow$ covariate shift.
Label Drift		Daily shift in output label distribution.	Any label varies by $> 15\%$ $\Rightarrow$ user behaviour change or label drift.
Mean Confidence		Average of all prediction confidence scores.	$\bar{c} < 0.75$ continuously $\Rightarrow$ domain shift, consider re-training.

9 Data Privacy and Model Robustness

9.1 Audit Logging and Privacy

All moderation decisions are logged to a JSONL audit file. To protect user privacy, the raw input text is never stored. Instead, the `log_event` tool computes `SHA-256(text)` and persists only the first 16 hexadecimal characters alongside metadata (timestamp, text length, language, predicted label, confidence, and action). Since SHA-256 is a cryptographic one-way function, the original text cannot be recovered from the log.

1. **Audit log anonymisation:** The `log_event` tool writes only the first 16 hexadecimal characters of the SHA-256 hash of the original text to the audit log. Since SHA-256 is a cryptographic one-way function, the original text cannot be recovered from the log, even if the log file is compromised.

9.2 Credential and Secret Management

All sensitive configuration values (HuggingFace API tokens, Ollama endpoint URLs) are stored exclusively in a local `.env` file that is declared in `.gitignore`. A template `.env.example` is committed instead, providing documentation without exposing secrets.

9.3 Adversarial Robustness

Hate speech detectors are prime targets for adversarial evasion: motivated users deliberately obfuscate toxic content to bypass automated systems. The model addresses three primary attack classes:

1. **Typographic obfuscation:** Replacing diacritics or inserting punctuation (e.g., `đ.ð m.ð`). The byte-level BPE tokeniser of XLM-R handles these gracefully; diacritic-removed copies of toxic texts are also injected into the training set via the diacritic augmentation pipeline.
2. **Teencode and abbreviations:** The preprocessing pipeline normalises common Vietnamese internet abbreviations before tokenisation, preventing evasion via spelling substitution.
3. **Implicit sarcasm without explicit slurs:** This is addressed by the agentic layer. When the classifier expresses uncertainty (confidence < 0.65), the Qwen2.5 agent analyses semantic context to catch sarcastic or culturally implicit hate speech that the discriminative classifier cannot resolve.

The robustness test suite (`src/training/robustness_cases.py`) contains 150+ handcrafted adversarial test cases spanning diacritic attacks, contrastive pairs, and sarcasm examples. These are automatically run as part of the `pytest` test suite.

10 Project Management and Teamwork

10.1 Project Timeline

The project was executed across six sequential phases, each with defined deliverables and milestones. Table 5 summarises the schedule.

Table 5: Project Development Timeline

Phase	Milestone	Key Deliverable	Duration
Phase 1	Data sourcing & problem definition	ViHSD CSV, preprocessing pipeline	Week 1–2
Phase 2	EDA & baseline modelling	TF-IDF + LR baseline, data quality report	Week 3
Phase 3	Deep learning core	Fine-tuned XLM-R Base, evaluation report	Week 4–5
Phase 4	Server infrastructure	FastAPI + Docker deployment	Week 6
Phase 5	Interactive GUI	Streamlit + Gradio dashboards	Week 7
Phase 6	Agentic AI & CL	Ollama agent, two CL rounds	Week 8

10.2 Task Breakdown and Role Simulation

For a solo or small team project, the following functional roles were simulated:

Table 6: Team Role Assignment

Member	Role	Key Responsibilities
Pham Minh Thong	ML/NLP Engineer	Model training, fine-tuning, CL pipeline, HuggingFace
Nguyen Trinh Quang	Backend Developer	FastAPI server, Docker, deployment, hot-reload
Dao Quoc Tuan	Data Engineer	Preprocessing, augmentation, data quality, EDA
Nguyen Le Tan Tien	UI & Evaluation	Streamlit GUI, Gradio, evaluation metrics, calibration

10.3 Reflection on Team Scaling

In a production engineering team of 5–10 engineers, the project’s modular architecture would support clean role separation. The `src/` package structure separates concerns (API, agent, data, training, evaluation) such that multiple developers can work in parallel with minimal merge conflicts. The structured YAML configuration system (`configs/`) allows ML engineers to iterate on model parameters without modifying any application code. The CL pipeline, triggered by a Google Drive success marker file, is deliberately designed to be triggered asynchronously by an MLOps engineer on a GPU node while the serving API continues running uninterrupted.

11 Ethics and Responsible AI

11.1 Ethics Impact Statement

Automated content moderation systems function as gatekeepers determining what speech is amplified or suppressed. While filtering hate speech is socially important, over-zealous classifiers risk censoring legitimate political criticism or minority dialects. Detailed ethical analysis is provided in `docs/ethics_statement.md`.

11.2 Beneficiaries and Harm Risks

Beneficiaries Online users are protected from targeted harassment and psychological harm. Marginalised groups benefit most directly, as hate campaigns targeting them are filtered automatically. Social media platforms benefit from reduced moderation overhead and legal risk.

Harm risks *False positives* may silence legitimate users, particularly those whose Southern or Central Vietnamese dialects contain slang misclassified as offensive. *False negatives* allow genuinely harmful content through, potentially enabling harassment campaigns. In the worst case, the system could be repurposed by platform operators to suppress political dissent under the guise of toxicity filtering.

11.3 Bias and Fairness Mitigation

Three concrete technical measures reduce systematic bias:

1. **Asymmetric inference thresholds:** Calibrated class-specific thresholds $\{\text{HATE} \geq 0.32, \text{OFFENSIVE} \geq 0.38\}$ increase minority class recall, reducing the risk of under-detection of genuine hate speech.
2. **Agentic second-pass validation:** The LLM agent acts as a high-precision second reviewer for uncertain cases, reducing false positives and false negatives simultaneously.
3. **Human-in-the-Loop (HITL) mechanism:** The Streamlit dashboard includes a review workflow where human moderators can correct model labels, feeding corrections back into the rehearsal buffer for the next CL cycle.

11.4 Explainability for Non-Technical Stakeholders

Two mechanisms improve transparency:

- **Soft probability visualisation:** The Streamlit dashboard displays the full class probability distribution as a Plotly bar chart, making the model's uncertainty immediately visible to administrators.

- **Agentic natural-language explanation:** When the LLM agent is triggered, it generates a Vietnamese-language explanation (e.g., “*Câu nói sử dụng từ ngữ thô tục để chửi bới, hạ nhục đối phương trực tiếp*”) that is displayed alongside the final classification, enabling non-technical stakeholders to audit individual decisions.

11.5 Taxonomy Safeguard Against Misuse

The system’s three-class taxonomy provides an important safeguard: HATE is defined strictly as targeted hostility towards protected characteristics (race, religion, gender, sexual orientation), while OFFENSIVE covers general vulgarity. This separation ensures that political criticism, consumer complaints, and legitimate dissent—which may contain strong language—are not automatically classified as hate speech.

12 Conclusion

This report has presented ViHSD, a complete end-to-end Vietnamese hate speech and toxicity detection system designed to address a pressing real-world content moderation problem. The project demonstrates that a carefully engineered NLP pipeline—combining domain-appropriate model selection, multi-strategy data augmentation, calibrated inference, agentic reasoning, and continual learning—can achieve competitive performance on a difficult three-class classification task in a low-resource language.

The core XLM-RoBERTa Base classifier achieves a Macro F1 of 64.61% and accuracy of 86.74% on the ViHSD benchmark, outperforming both the classical baseline and PhoBERT. The hybrid agentic architecture—routing borderline cases to a local Qwen2.5-7B LLM—improves handling of sarcasm and implicit hate speech while limiting LLM invocation to approximately 10–15% of traffic. Two rounds of continual learning on the VLSP-2019 corpus demonstrate the model’s adaptability to new data domains, achieving a Macro F1 of 70.97% on the mixed evaluation set after the second round while maintaining CLEAN class F1 above 95%.

Limitations and future work.

- The OFFENSIVE class remains the most challenging, with F1 of 41.13%. Future work should investigate richer OFFENSIVE-specific training data and more granular sub-class distinctions.
- The LLM agent currently processes one text at a time. Batch LLM inference (e.g., via structured parallel prompting) would significantly reduce latency in batch moderation workflows.
- The VLSP-2019 CL results are evaluated on a mixed validation set that includes training domain samples; a full held-out VLSP test set evaluation would provide a cleaner measure of domain transfer performance.
- Longer-term deployment would benefit from active learning: automatically flagging the highest-uncertainty predictions for human annotation and incorporating corrections into the next CL cycle.

12 References

- [1] Luu, S. T., Nguyen, K. V., and Nguyen, N. L. T. (2021). ViHSD: A Human-Annotated Dataset for Detecting Hate Speech on Vietnamese Social Media. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7530–7542.
- [2] VLSP Shared Task Committee (2019). VLSP 2019: Vietnamese Hate Speech Detection Shared Task. Vietnam Language and Speech Processing (VLSP) Workshop.
- [3] Conneau, A., Khandelwal, K., Goyal, N., et al. (2020). Unsupervised Cross-lingual Representation Learning at Scale. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8440–8451.
- [4] Nguyen, D. Q. and Nguyen, A. T. (2020). PhoBERT: Pre-trained Language Models for Vietnamese. *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1037–1042.
- [5] Lin, T. Y., Goyal, P., Girshick, R., He, K., and Dollár, P. (2017). Focal Loss for Dense Object Detection. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988.
- [6] Kirkpatrick, J., Pascanu, R., Rabinowitz, N., et al. (2017). Overcoming Catastrophic Forgetting in Neural Networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521–3526.
- [7] Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330.
- [8] Qwen2 Team (2024). Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671*.